

KernelTileSpec — a field-by-field guide

teenygrad tile metadata, with worked examples on real kernel signatures

22 September 2026

Contents

1. What the types are for, in one paragraph	2
2. The six types	2
2.1 The single most important idea: names, not arithmetic	2
2.2 Unresolved means full extent	3
2.3 Coverage is opt-in	3
3. Field reference	4
TensorTileSpec::rank versus axes.len()	4
TileAxisBinding::dims — a list, not an index	4
block_const and extent_param — both are names	4
reduction_axis	4
untiled_dims — documentation, with a caveat	4
divide_by	4
TileWindow — three names, and no padding in the answer	5
TileLoopSpec / TileCarryBinding	5
4. Worked examples — specs that exist today	6
4.1 Flat elementwise — derived, not hand-authored	6
4.2 Two tiled axes plus a reduction — MATMUL_TILE_SPEC	6
4.3 A flattened binding — BATCHNORM2D_TILE_SPEC	6
4.4 An input with no bindings at all — CONV1D_TILE_SPEC	7
4.5 Attributes on a real multi-axis kernel — conv2d_forward	7
5. Worked proposals — the three types with no usage yet	9
5.1 TileWindow on conv2d_forward — teenygrad-1nr.29	9
5.2 divide_by on group_norm_forward — teenygrad-1nr.15	9
5.3 TileLoopSpec on flash_attention2_forward — teenygrad-1nr.12	10
6. Gotchas	11
7. Where things live	11
8. References	11
Papers	11
Documentation	12
Issues referenced	12

1. What the types are for, in one paragraph

A `KernelTileSpec` is **static, declarative metadata** describing how one kernel’s tensors are sliced into tiles. It is data, not code: it never affects what is generated into a kernel body. Its consumer is the scheduler — Welder’s `Propagate` — which starts from a chosen output tile and walks backwards through the graph asking, at each node, “if this node’s output is tiled *this* way, what tile does each input need?” The spec is how a kernel answers that question without the scheduler knowing anything about the kernel’s body.

Everything lives in `core/teeny-core/src/model/tile_spec.rs` (250 lines, six types, no dependencies). Specs are attached per-op by `TritonLowering` in `kernels/teeny-kernels/src/graph/mod.rs`, either as hand-authored `consts` or derived by `#[tiled_kernel]` from `#[tile(...)]` attributes on the kernel’s own parameters.

2. The six types

```
KernelTileSpec {                                // one kernel
    inputs:    &'static [TensorTileSpec],
    outputs:   &'static [TensorTileSpec],
    loop_spec: Option<TileLoopSpec>,
}

TensorTileSpec {                                // one tensor (one pointer param)
    param:      &'static str,
    rank:       usize,
    axes:       &'static [TileAxisBinding],
    reduction_axis: Option<usize>,
    untiled_dims: &'static [&'static str],
}

TileAxisBinding {                               // one axis of one tensor
    dims:      &'static [usize],
    block_const: &'static str,
    extent_param: &'static str,
    window:     Option<TileWindow>,
    divide_by:  Option<usize>,
}

TileWindow {                                    // a strided/padded read
    stride_const:    &'static str,
    pad_const:       &'static str,
    kernel_size_const: &'static str,
}

TileLoopSpec { carries: &[TileCarryBinding], trip_count_factors: &[&str] }
TileCarryBinding { name: &'static str, shape_consts: &'static [&'static str] }
```

Note what these are **not**: there are no enums and no variants. Every “case” below is a *combination of field values*, which is why the shapes are easier to learn from examples than from the type definitions.

2.1 The single most important idea: names, not arithmetic

“Every axis declares an `extent_param` name. Two axes anywhere — on the same tensor, on different tensors, on an input or an output — that declare the same `extent_param` name are the same free variable.” — module doc

Propagation is **name matching**. Resolving one output axis by name resolves every other axis that shares that name, on any tensor, with no arithmetic involved. A GEMM's `a_ptr` and `c_ptr` both declaring "M" means propagating `c_ptr`'s chosen M resolves `a_ptr`'s M for free.

The corollary matters just as much: **an axis whose name never appears in the output is correctly left unresolved**. A GEMM's reduction axis "K" appears on `a_ptr` and `b_ptr` but never on `c_ptr`, so nothing propagates to it. That is not a gap — it matches Welder's model, where a reduction axis's tile size is chosen by the tiling search rather than propagated top-down.

2.2 Unresolved means full extent

An axis that resolves to nothing keeps its **full extent**. This is the conservative direction — an over-estimate is safe for a memory footprint checked against a hard capacity; an under-estimate is not — and it is the documented fallback for three separate situations: an axis omitted from `axes`, an axis named in `untiled_dims`, and an axis whose `extent_param` never matched.

2.3 Coverage is opt-in

Most ops have no spec at all. `ExecutableOp::tile_spec()` returns `None` by default, and `Propagate` treats a missing spec as a **hard boundary** — it stops there rather than guessing. Only four hand-authored specs plus three derived ones exist today.

3. Field reference

`TensorTileSpec::rank` versus `axes.len()`

`rank` is the tensor’s **real** rank — how many dims the actual graph edge carries. `axes` may describe **fewer** dims than `rank`, and frequently does. Every dim without a binding keeps its full extent.

This is the field most likely to mislead: `axes.len()` is not the rank, and nothing requires the two to line up. `CONV1D_TILE_SPEC`’s input has `rank: 3` and `axes: &[]` — it describes none of its dims, deliberately.

`TileAxisBinding::dims` — a list, not an index

Almost always one entry. More than one entry means this binding’s block spans a **flattened** run of several real axes: a kernel iterating a combined `H*W` range in one `BLOCK_HW`-wide loop writes `dims: &[2, 3]`.

When a flattened binding is resolved, the block lands on the **last (innermost)** entry and every other entry is set to 1. That is product-preserving — the tile’s element count still matches `BLOCK_HW` — rather than a literal axis-aligned subregion. Two invariants, both authoring bugs if broken, and both silently tolerated by the consumer:

- every index must be $< \text{rank}$ — an out-of-range index is **silently skipped**
- no index may repeat across one tensor’s `axes` — a repeat is **last-write-wins**

`KernelTileSpec::validate()` exists specifically because neither of those surfaces on its own.

`block_const` and `extent_param` — both are names

`block_const` names the `const NAME: i32` generic giving the tile size. `extent_param` names the `NAME: i32` runtime parameter giving the axis’s total extent. **Neither carries a value.** The metadata has no per-node parameter lookup, so a consumer that needs actual numbers must be handed a resolver by its caller.

`reduction_axis`

Documents which axis this tensor is reduced/accumulated over. It does not by itself change resolution — the axis goes unresolved because its name has no output-side counterpart, not because it is flagged here.

`untiled_dims` — documentation, with a caveat

Names the params giving the sizes of dims that are real but not individually tiled — a conv output’s batch, channel and outer spatial axes, which are grid-driven. The strings are **not read** by the consumer. The effect they describe happens structurally, from `rank` and `axes`: a dim omitted from `axes` keeps its full extent whether or not it is named here.

So this list need not be complete, and in `CONV1D_TILE_SPEC` it is empty even though every dim is untiled.

`divide_by`

Some(`g`) when the axis’s usable per-tile extent is its full extent divided by a compile-time constant `g` that is not named anywhere else — GroupNorm’s `channels_per_group = C / G`, where `G` is an internal kernel constant rather than a graph-level dimension.

It is a **plain number, not a name**, precisely because the divisor’s value must be known when the spec is constructed. Consequently an op whose divisor varies per instance must build its `tile_spec()` fresh per call from its own instance data, and cannot return one shared `const`.

Its contract is to replace the axis’s **raw full extent** wherever that would be used — not to divide a block propagated from the output.

TileWindow — three names, and no padding in the answer

Describes an axis read through a strided/padded sliding window rather than a contiguous slice. Given an output tile of `block` elements, the input region read is the **receptive field**:

$$\text{input extent} = (\text{block} - 1) \times \text{stride} + \text{kernel}$$

Padding does not appear. It shifts the origin of the region, not its size. This is the single most common place to go wrong, and it is worth stating why: the whole-tensor relation between input and output extents *does* involve padding,

$$\text{out} = \left\lfloor \frac{\text{in} + 2p - k}{s} \right\rfloor + 1,$$

so inverting it gives $\text{in} = s(\text{out}-1) + k - 2p$. That $-2p$ term is correct only when the “tile” is the entire output — the one case where both ends of the region sit on the tensor’s padded edges. An interior tile touches no padding at all. A 3×3 convolution with $p=1$, $s=1$ and a block of 8 reads **10** input elements; the inverted-extent formula says 8.

Two different questions, with two different answers:

question	answer
which input <i>extent</i> produced this output extent?	many-to-one; s answers
which input <i>elements</i> does this output tile read?	exactly one answer

Tiling asks the second. It therefore inherits none of the first’s ambiguity.

TileLoopSpec / TileCarryBinding

Some when the kernel’s body loops with carried accumulator state instead of resolving its whole output in one shot. `carries` names the kernel-body-local tensors updated in place across iterations, with their shapes given as const names. `trip_count_factors` names the params determining the trip count — as documentation, **not a formula to evaluate**: `conv2d`’s real trip count is $(C_IN/G) \cdot KH \cdot KW$, one runtime param combined with three consts.

Declaring `loop_spec` records the loop’s existence and carried shape and nothing more. Per its own doc comment, the consumer still treats a loop-carried kernel’s real inputs as fully materialised.

4. Worked examples — specs that exist today

Every spec in this section is quoted verbatim from `kernels/teeny-kernels/src/graph/mod.rs`.

4.1 Flat elementwise — derived, not hand-authored

```
#[tilde_kernel]
pub fn relu_forward<T: Triton, D: Num, const BLOCK_SIZE: i32>(
    #[tile(block = BLOCK_SIZE, extent = n_elements)] x: In<Tile<T, D>>,
    #[tile(block = BLOCK_SIZE, extent = n_elements)] y: Out<Tile<T, D>>,
    n_elements: i32,
)
```

The simplest shape: one tiled axis, input tile equals output tile, no window, no reduction, no flattening. `#[tilde_kernel]` derives `ReluForward::tile_spec(rank)` from those attributes, so the spec cannot drift from the signature — rename `n_elements` without updating the attribute and it fails to compile.

Note the spec is a **function of rank**, not a `const`: the same flat kernel serves a rank-2 and a rank-4 tensor.

4.2 Two tiled axes plus a reduction — MATMUL_TILE_SPEC

```
pub fn matmul_forward<T, D, const BLOCK_M: i32, const BLOCK_N: i32,
    const BLOCK_K: i32, const GROUP_M: i32>(
    a_ptr: In<T::Pointer<D>>, b_ptr: In<T::Pointer<D>>,
    c_ptr: InOut<T::Pointer<D>>, M: i32, N: i32, K: i32,
)

const A: TensorTileSpec = TensorTileSpec {
    param: "a_ptr", rank: 2,
    axes: &[
        TileAxisBinding { dims: &[0], block_const: "BLOCK_M",
            extent_param: "M", window: None, divide_by: None },
        TileAxisBinding { dims: &[1], block_const: "BLOCK_K",
            extent_param: "K", window: None, divide_by: None },
    ],
    reduction_axis: Some(1),
    untiled_dims: &[],
};
// B: same shape, K on dim 0 and N on dim 1, reduction_axis: Some(0)
// C: M on dim 0, N on dim 1, reduction_axis: None
KernelTileSpec { inputs: &[A, B], outputs: &[C], loop_spec: None }
```

Read it by name. `c_ptr` declares "M" and "N". Propagating an output tile of [64, 32] resolves `M` = 64 and `N` = 32, which immediately resolves `a_ptr`'s dim 0 and `b_ptr`'s dim 1. "K" appears only on the inputs, so it stays unresolved and both operands keep their full K extent:

output tile [64, 32] -> `a_ptr` [64, K_full] `b_ptr` [K_full, 32]

4.3 A flattened binding — BATCHNORM2D_TILE_SPEC

The kernel's grid is [C, B], one CTA per channel×batch, each looping the flattened H×W range in `BLOCK_HW`-wide tiles. No single real axis corresponds to `BLOCK_HW`:

```
const HW: TileAxisBinding = TileAxisBinding {
    dims: &[2, 3], // H and W, W innermost (NCHW row-major)
```

```

    block_const: "BLOCK_HW",
    extent_param: "HW",
    window: None,
    divide_by: None,
};
const X: TensorTileSpec = TensorTileSpec {
    param: "x_ptr", rank: 4, axes: &[HW],
    reduction_axis: None,
    untiled_dims: &["B", "C"],    // real, grid-driven, not individually tiled
};
const Y: TensorTileSpec = TensorTileSpec { param: "y_ptr", ..X };

```

Resolving HW = 256 gives the tile [None, None, 1, 256]: the block lands on the innermost entry, the outer entry collapses to 1, and dims 0–1 stay at full extent because they have no binding. Because it is shape-preserving, `x_ptr` and `y_ptr` share the same "HW" name, so the output resolves the input directly.

4.4 An input with no bindings at all — CONV1D_TILE_SPEC

```

const X: TensorTileSpec = TensorTileSpec {
    param: "x_ptr", rank: 3, axes: &[],    // nothing described
    reduction_axis: None, untiled_dims: &[],
};
const Y: TensorTileSpec = TensorTileSpec {
    param: "y_ptr", rank: 3,
    axes: &[TileAxisBinding { dims: &[2], block_const: "BLOCK_OL",
        extent_param: "OL", window: None, divide_by: None }],
    reduction_axis: None, untiled_dims: &[],
};

```

Input and output share **no** axis name, unlike the shape-preserving cases. The input therefore resolves to [None, None, None] — every dim at full extent.

That is correct but pessimistic, and it is exactly the gap `TileWindow` is meant to close: the real input region for a `BLOCK_OL`-wide output strip is the receptive field, not the whole axis. Until `teenygrad-1nr.29` populates the window, conv and pool over-report their input tiles.

4.5 Attributes on a real multi-axis kernel — conv2d_forward

```

#[tile(name = "B",    extent = _B)]
#[tile(name = "C_IN", extent = C_IN)]
#[tile(name = "H",    extent = H)]
#[tile(name = "W",    extent = W)]
x_ptr: In<T::Pointer<D>>,
w_ptr: In<T::Pointer<D>>,
#[tile(name = "B",    extent = _B)]
#[tile(name = "C_OUT", extent = C_OUT)]
#[tile(name = "OH",    extent = OH)]
#[tile(name = "OW",    block = BLOCK_OW, extent = OW)]
y_ptr: Out<T::Pointer<D>>,

```

Two things to read here. First, an attribute **without** `block` declares an untiled axis — it still names the axis so the scheduler knows the dim exists and what its extent param is called. Only `OW` carries a `block`, because only the innermost spatial axis is genuinely block-tiled; batch, channels and `OH` are grid-driven.

Second, `w_ptr` has no attributes at all, so it is absent from the spec entirely: the weight is not tiled by this kernel's grid.

The attribute is **metadata-only**. It drives `tile_spec()` and `grid_spec()` generation and never touches the body — `x_ptr/y_ptr` stay plain pointers and the kernel's hand-written `pid` decode is untouched. This is deliberate: an earlier design also spliced index arithmetic into the body, and that coupling is what made it break under composition and get removed.

5. Worked proposals — the three types with no usage yet

Nothing in this section is in the tree. `TileWindow`, `divide_by` and `TileLoopSpec` are declared and documented but populated by no shipped spec. These are written against each kernel’s real signature to show the shape each type is meant to take, and each names the issue that would land it.

5.1 `TileWindow` on `conv2d_forward` — `teenygrad-1nr.29`

The kernel’s real consts are `KH`, `KW`, `STRIDE_H`, `STRIDE_W`, `PAD_H`, `PAD_W`, `G`, `BLOCK_OW`. The `W` axis of `x_ptr` is read through a window driven by the `OW` axis of `y_ptr`:

```
// PROPOSED - not in the tree. teenygrad-1nr.29.
TileAxisBinding {
    dims:          &[3],           // W, on x_ptr
    block_const:   "BLOCK_OW",     // the driving output block
    extent_param:  "W",
    window: Some(TileWindow {
        stride_const: "STRIDE_W",
        pad_const:    "PAD_W",
        kernel_size_const: "KW",
    }),
    divide_by: None,
}
```

With `BLOCK_OW = 8`, `KW = 3`, `STRIDE_W = 2`, this resolves `x_ptr`’s `W` axis to $(8-1) \cdot 2 + 3 = 17$, **whatever** `PAD_W` is. A resolved extent that changes with the padding value is the signature of the inverted-extent mistake from §3.

Two open design points this proposal does not settle. The consts are *names*, and the metadata has no value lookup, so a consumer needs a caller-supplied resolver — the same limitation `divide_by` hit and solved by baking in a plain number at construction time. And `#[tile(...)]` cannot currently express a window at all; today it accepts `name`, `block` and `extent` only.

5.2 `divide_by` on `group_norm_forward` — `teenygrad-1nr.15`

```
pub fn group_norm_forward<T: Triton, D: Float, const BLOCK_NL: i32>(
    x_ptr: In<T::Pointer<D>>, y_ptr: Out<T::Pointer<D>>,
    weight_ptr: In<T::Pointer<D>>, bias_ptr: In<T::Pointer<D>>,
    _N: i32, C: i32, L: i32, G: i32, eps: f32,
)
```

The grid is one CTA per (n, g) . Each CTA covers `channels_per_group * L` elements, where `channels_per_group = C / G`, read as one flattened `BLOCK_NL`-wide range. So this needs **both** mechanisms at once — flattening *and* a divisor:

```
// PROPOSED - not in the tree. teenygrad-1nr.15.
TileAxisBinding {
    dims:          &[1, 2],       // C and L, jointly, L innermost
    block_const:   "BLOCK_NL",
    extent_param:  "CL",
    window:        None,
    divide_by:     Some(/* G, resolved at tile_spec() construction */),
}
```

Note `divide_by` cannot be written as a literal here. `G` is a runtime parameter of this kernel, but `divide_by` must be a plain number known when the spec is built — so `GroupNormForward` must construct its `tile_spec()` per

instance from its own `num_groups`, not return a shared `const`. That is stated in the field’s own doc comment and is the main reason this spec has not been written yet.

`teenygrad-1nr.15` also records that the existing regression test for `divide_by` deliberately modelled the channel axis and `L` as two *separate* bindings, which is **not** what the real kernel does — it reads them jointly. The two mechanisms have never been combined and tried against the real kernel.

5.3 TileLoopSpec on `flash_attention2_forward` — `teenygrad-1nr.12`

```
pub fn flash_attention2_forward<T: Triton, D: Float, const HEAD_DIM: i32>(
  q_ptr: In<...>, k_ptr: In<...>, v_ptr: In<...>,
  o_ptr: Out<...>, l_ptr: Out<...>,
  n_ctx_q: i32, n_ctx_k: i32, softmax_scale: f32, neg_inf: f32,
)
```

The body runs a real online-softmax loop, for `k_row` in `0..n_ctx_k`, carrying three `[HEAD_DIM]` tensors updated in place:

```
let mut acc = T::zeros::<D>(&[HEAD_DIM]);
let mut m_i = T::full(&[HEAD_DIM], D::from_f64(neg_inf as f64));
let mut l_i = T::zeros::<D>(&[HEAD_DIM]);
```

// PROPOSED - not in the tree. teenygrad-1nr.12.

```
TileLoopSpec {
  carries: &[
    TileCarryBinding { name: "acc", shape_consts: &["HEAD_DIM"] },
    TileCarryBinding { name: "m_i", shape_consts: &["HEAD_DIM"] },
    TileCarryBinding { name: "l_i", shape_consts: &["HEAD_DIM"] },
  ],
  trip_count_factors: &["n_ctx_k"],
}
```

This is a fixed-trip-count loop, so `trip_count_factors` names one param. Contrast `conv2d`, whose loop bound is $(C_IN/G) \cdot KH \cdot KW$ and would name `["C_IN", "G", "KH", "KW"]` — four factors, not a formula, because nothing evaluates them.

Declaring this would make the loop’s existence and carried shape queryable. It would **not**, on its own, stop `propagate/mem_traffic/mem_footprint` treating `k_ptr` and `v_ptr` as fully materialised — that needs a consumer that understands loop-carried state.

6. Gotchas

axes.len() is not the rank. Read `rank`. A tensor may describe some, all or none of its dims.

A malformed spec fails silently. An out-of-range `dims` index is skipped; a repeated one is last-write-wins. Neither raises anything. Run `KernelTileSpec::validate()`.

untiled_dims is inert. The strings are never read. Omitting a dim from `axes` is what actually produces the full-extent fallback; naming it here is commentary.

Names are the whole propagation mechanism. A typo in an `extent_param` does not error — it silently creates a *new* free variable that nothing resolves, and the axis quietly falls back to full extent.

Nothing carries a value. `block_const`, `extent_param` and all three `TileWindow` fields are names of consts and params. A consumer needing numbers must be given a resolver.

Padding never enters a tile extent. Only the origin.

Hand-authored specs can drift. Nothing enforces that `MATMUL_TILE_SPEC` matches `matmul_forward`’s real parameters; the derived ones cannot drift, which is the argument for `teenygrad-1nr.18`.

There is no consumer in the tree right now. `TileGraph::propagate` was deleted by the v2 rewrite and is being re-landed by `teenygrad-1nr.25`. Until then a spec can be written, and be wrong, with nothing to notice.

7. Where things live

what	where
the six types	<code>core/teeny-core/src/model/tile_spec.rs</code>
hand-authored specs	<code>kernels/teeny-kernels/src/graph/mod.rs</code>
<code>#[tile(...)]</code> parsing / spec derivation	<code>macros/teeny-macros/src/macros/tiled_kernel.rs</code>
the opt-in hook	<code>ExecutableOp::tile_spec()</code> , <code>core/teeny-core/src/model/mod.rs</code>
sibling grid metadata	<code>core/teeny-core/src/model/grid_spec.rs</code>

8. References

Papers

- Yining Shi et al., “**Welder: Scheduling Deep Learning Memory Access via Tile-graph**”, OSDI ’23. The source of `Propagate`, the tile-graph model and the memory-traffic cost model this metadata serves. <https://www.usenix.org/conference/osdi23/presentation/shi>
- Hongyu Zhu et al., “**ROLLER: Fast and Efficient Tensor Compilation for Deep Learning**”, OSDI ’22. The tile-shape enumeration `Welder’s EnumerateSubtiles` builds on — power-of-two candidates aligned to hardware features. <https://www.usenix.org/conference/osdi22/presentation/zhu>
- Lingxiao Ma et al., “**Rammer: Enabling Holistic Deep Learning Compiler Optimizations with rTasks**”, OSDI ’20. Whole-graph scheduling; `Welder` is implemented on top of it. <https://www.usenix.org/conference/osdi20/presentation/ma>
- Tianqi Chen et al., “**TVM: An Automated End-to-End Optimizing Compiler for Deep Learning**”, OSDI ’18. `Welder`’s kernel generation is built on `TVM`. <https://www.usenix.org/conference/osdi18/presentation/chen>
- Philippe Tillet et al., “**Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations**”, MAPL ’19. The tile-level programming model these kernels are written against. <https://dl.acm.org/doi/10.1145/3315508.3329973>

- Zhen Zheng et al., “**AStitch: Enabling a New Multi-dimensional Optimization Space for Memory-Intensive ML Training and Inference**”, ASPLOS ’22. The alternative memory-intensive fusion approach Welder is measured against. <https://dl.acm.org/doi/10.1145/3503222.3507723>
- Tri Dao, “**FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning**”, 2023. The algorithm behind the loop-carried kernel in §5.3. <https://arxiv.org/abs/2307.08691>

Documentation

- Triton language documentation — <https://triton-lang.org/>
- Welder artifact (OSDI ’23) — <https://github.com/microsoft/nnfusion/tree/welder>

Issues referenced

`teenygrad-1nr` (Welder-style TileGraph scheduler) and its children — `1nr.12` loop-carried kernels, `1nr.15` GroupNorm, `1nr.18` deriving specs from signatures, `1nr.25` re-landing `propagate`, `1nr.29` windowed `#[tile(...)]`, `1nr.30` receptive-field propagation. `teenygrad-1iy` covers spec coverage and isolated testing.